WOXSEN UNIVERSITY

School of Technology

KAIROS: An Autonomous Causal Business Intelligence Agent for Proactive Multi-Source Metric Monitoring and Root Cause Analysis

Capstone Project-I Report

Submitted in partial fulfilment of the requirements for the degree of

Bachelor of Technology in Computer Science and Engineering

Submitted by

Palla Chetana Reddy (Roll No. to be added)

Varshini Yadav Nasaram (Roll No. to be added)

Obulagari Nandini (Roll No. to be added)

Under the guidance of

Dr. K RajKumar, Faculty designation to be confirmed

Department of Computer Science and Engineering

Woxsen University, School of Technology, Hyderabad, Telangana

Academic Year 2026–2027 | 7th Semester

ABSTRACT

Many business intelligence systems require a person to notice a problem and open a dashboard, while correlation-based explanations do not establish which metric moved first. This project presents KAIROS, a five-agent pipeline for scheduled monitoring of business metrics from heterogeneous sources. Protocol-level adapters collect SQL, file, SFTP/FTP, REST, and pushed data; aggregation can run near its source, and a resumable chunked service transfers raw files when necessary. A seasonal median and median-absolute-deviation detector, optionally combined with Prophet, identifies anomalies. DYNOTEARS learns a lagged causal graph, and a fact-grounded reporter checks numeric and metric-name claims before alerts are delivered. Across ten synthetic benchmark seeds, DYNOTEARS achieved F1 = 0.97 ± 0.02 and identified both injected incident origins in every run. In a localhost transfer benchmark with a 50-Mbit/s per-connection cap, eight-stream transfer was 6.4 times faster than single-stream raw FTP; edge aggregation reduced a 105.6-MB export to 192.7 KB. These results demonstrate feasibility under controlled conditions, not validated production accuracy. Evaluation on real business data remains future work.

Keywords: proactive business intelligence, anomaly detection, causal discovery, DYNOTEARS, root cause analysis, multi-agent systems, LLM verification, data ingestion.

TABLE OF CONTENTS

1. Introduction 1
1.1 Background 1
1.2 Problem Statement 1
1.3 Significance 2
1.4 Scope 2
1.5 Organisation of the Report 2
2. Technology Review 3
2.1 Review Method 3
2.2 Related Technologies 3
2.3 Comparison and Analysis 5
2.4 Review Summary 5
2.5 Research Gap 6
3. Objectives and Methodology 7
3.1 Project Objectives 7
3.2 Proposed Approach 7
3.3 System Architecture 7
3.3.1 Agent 1: Data Collector 8
3.3.2 Agent 2: Anomaly Watchman 9
3.3.3 Agent 3: Causal Investigator 9
3.3.4 Agent 4: NL Reporter 9
3.3.5 Agent 5: Messenger 9
3.3.6 Ingestion Strategy for Any Source 9
3.4 Tools and Technologies 11
3.5 Implementation Plan 11
3.6 Novelty 12
4. Implementation and Results 13
4.1 Implementation Details 13
4.2 System Features 13
4.3 Testing and Validation 15
4.4 Results and Analysis 16
4.4.1 Multi-Source Ingestion 16
4.4.2 Large-File Transfer Benchmark 16
4.4.3 Anomaly Detection 17
4.4.4 Causal Discovery and Root Cause 18
4.4.5 Report Verification 20

4.5 Discussion 21
5. Conclusion and Future Work 22
5.1 Summary 22
5.2 Objectives Achievement 22
5.3 Limitations 22
5.4 Lessons Learned 22
5.5 Recommendations 23
5.6 Future Scope for Capstone Project-II 23
Acknowledgment 24
References 25
Appendix A: Source Code Snippets 27
Appendix B: User Manual 28
Appendix C: Configuration and API Reference 29
Appendix D: Additional Results 30

LIST OF FIGURES

Figure 1: KAIROS system architecture 8
Figure 2: LangGraph state machine for one monitoring cycle 8
Figure 3: Ingestion routes by data location and size, with measured results 10
Figure 4: Fast Transfer protocol, including resume after a dropped connection 11
Figure 5: Project timeline (Gantt chart) for the 7th semester 12
Figure 6: Overview page after a monitoring cycle on live demo sources 13
Figure 7: Data Sources page with SQL, FTP, file, REST and push sources 14
Figure 8: Fast Transfer page with transfer history and benchmark results 15
Figure 9: Transfer time and bytes on the wire for each method 17
Figure 10: Detected anomalies on four metrics, coloured by severity 18
Figure 11: Precision, recall, F1 and structural Hamming distance of each method 18
Figure 12: Graph learned by DYNOTEARS (seed 3) compared with the true structure 19
Figure 13: Causal Analysis page with the CBMG and ranked root-cause chains 20
Figure 14: Reports page with an LLM-written report marked as verified 21

LIST OF TABLES

Table 1: Technology review: BI platforms and anomaly detection 3
Table 2: Technology review: causal analysis and agent orchestration 4
Table 3: Technology review: storage, ingestion and transfer 4
Table 4: Coverage of project requirements by existing approaches 5
Table 5: Research gaps identified from the technology review 6
Table 6: SMART objectives of the project 7
Table 7: Tools and technologies used 11
Table 8: Automated test suite 15
Table 9: Live ingestion demonstration 16
Table 10: Ingestion benchmark for a 105.6 MB raw export 16
Table 11: Anomaly detection over ten seeds (mean ± standard deviation) 17
Table 12: Causal discovery against ground truth over ten seeds 18
Table 13: LLM-Modulo verifier stress test 20
Table 14: Achievement of the SMART objectives 22

LIST OF ABBREVIATIONS

API	Application Programming Interface
BI	Business Intelligence
CBMG	Causal Business Metric Graph
CDC	Change Data Capture
CSV	Comma-Separated Values
DAG	Directed Acyclic Graph
DYNOTEARS	Dynamic NOTEARS (structure learning for time series)
ELT	Extract, Load, Transform
FTP / SFTP	File Transfer Protocol / SSH File Transfer Protocol
KPI	Key Performance Indicator
LLM	Large Language Model
MAD	Median Absolute Deviation
ProBI	Proactive Business Intelligence
RCA	Root Cause Analysis
REST	Representational State Transfer
SHD	Structural Hamming Distance
SLAC	Semantic Layer Auto-Construction
SMART	Specific, Measurable, Achievable, Relevant, Time-bound
SVAR	Structural Vector Autoregression
WAN	Wide Area Network

1. Introduction

1.1 Background

Most organisations now hold their operating data in several places at once: an ERP database for orders, spreadsheets kept by the customer-support team, CSV exports from marketing tools, and REST APIs offered by logistics or payment providers. Business intelligence (BI) platforms such as Microsoft Power BI and Tableau [2] bring this data together into dashboards, and they have become standard equipment for managers. Their working model has stayed the same for two decades, however. A person opens a dashboard, looks at a chart and decides whether something is wrong. If nobody looks, nothing is found.

Two developments make a different model possible. Time-series anomaly detection has matured to the point where seasonal business metrics can be monitored automatically [3], [4], [5]. Causal discovery methods such as NOTEARS and its time-series extension DYNOTEARS can learn directed, lagged relationships between variables from observational data [6], [7]. Large language models (LLMs) [8] can also turn numerical findings into readable text, although they are known to state numbers that are not in their input [9]. Agent frameworks built on the ReAct pattern [10] and graph-based orchestration [11] make it practical to chain these steps into one autonomous loop.

1.2 Problem Statement

Current BI practice has three linked weaknesses. First, monitoring is reactive: an incident that starts on a Friday evening may be seen only on Monday. Second, the explanation features that do exist, such as Key Influencers in Power BI or Explain Data in Tableau, rank correlated fields and cannot say which metric moved first or which one caused the other. Third, connecting the data is expensive. Managed ELT services charge by data volume, every company uses a different mix of systems, and moving large exports over FTP is slow. During the second evaluation of this project, reviewers specifically asked how data from any source would be transferred, given that connectors are heavy and costly and FTP is slow for large files. The problem addressed is therefore to build a system that monitors multi-source business metrics without human initiation, identifies the causal origin of an anomaly, explains it accurately, and ingests data from heterogeneous systems at negligible cost.

1.3 Significance

Small and mid-sized businesses benefit most, because they rarely have a data team watching dashboards around the clock. Earlier detection shortens the time between a fault, such as a paused advertising campaign or a warehouse delay, and the corrective action. A causal explanation saves analysts the manual work of tracing a chain of related metrics. Because KAIROS uses only free software and moves aggregated metrics instead of raw tables, its running cost stays close to zero.

1.4 Scope

The project covers numeric business metrics recorded as time series at hourly or finer resolution. It includes ingestion from SQL databases, SFTP/FTP servers, local and cloud files, REST APIs and push clients; anomaly detection; causal discovery and root-cause ranking; verified report generation; delivery through Telegram, e-mail and a web dashboard; and a transfer service for large files. Unstructured data such as text or images, real-time streaming below one-minute latency, and multi-tenant deployment are outside the scope of Capstone Project-I. Evaluation uses a synthetic benchmark with a known causal structure, because real company data with labelled root causes was not available within the semester.

1.5 Organisation of the Report

Chapter 2 reviews existing technologies and identifies the research gap. Chapter 3 states the objectives and describes the methodology, architecture and novelty. Chapter 4 presents the implementation, testing and results. Chapter 5 concludes with limitations, lessons learned and the plan for Capstone Project-II.

2. Technology Review

2.1 Review Method

Twelve technologies were reviewed across the four functions KAIROS needs: analytics and anomaly detection, causal analysis and orchestration, and data storage and transfer. Each was assessed on the eleven criteria set by the project guidelines, using vendor documentation, the original research papers and hands-on trials where the software was freely available. Similar technologies are compared side by side in Table 1 to Table 3.

2.2 Related Technologies

Table 1: Technology review: BI platforms and anomaly detection

Criterion	Power BI	Tableau	Prophet	Anomaly Transformer
Description	Microsoft BI and reporting platform with desktop and cloud editions [2].	Visual analytics platform owned by Salesforce.	Open-source additive forecasting model from Meta [3].	Transformer for unsupervised time-series anomaly detection [1].
Key features	Power Query connectors, DAX, Key Influencers, per-chart anomaly markers.	Drag-and-drop visuals, Explain Data, Pulse metric digests.	Trend, daily and weekly seasonality, holidays, uncertainty intervals.	Association discrepancy, minimax training.
Performance	In-memory VertiPaq engine; interactive on millions of rows.	Hyper engine; fast extracts.	Seconds per series on a laptop.	Strong results on public benchmarks.
Scalability	Premium capacity scales to large datasets.	Server and Cloud deployments.	One model per series; parallelisable.	Needs GPU training per dataset.
Cost / licensing	Desktop free; Pro and Premium per-user subscriptions.	Creator, Explorer and Viewer subscriptions.	Free, MIT licence.	Free research code.
Compatibility	Windows desktop, web, mobile; hundreds of connectors.	Windows, macOS, web; many connectors.	Python and R; Stan backend.	Python, PyTorch.
Ease of use	Low for charts; DAX has a steep curve.	Moderate; strong community.	Easy; few parameters.	Hard; research-grade code.
Security	Azure AD, row-level security, encryption.	SSO, row-level security.	Library only; no network surface.	Not applicable.
Current market	Repeatedly named a Leader in Gartner's Analytics and BI Magic Quadrant.	Widely used in enterprises.	Very popular for business forecasting.	Academic use.
Limitations	Reactive; explanations are correlational; anomaly markers are per visual.	Explain Data lists candidate factors, not causal chains.	Univariate; slower than statistical scores.	Needs long training windows and tuning per metric.
Relevance to project	Baseline of reactive BI; not used.	Baseline; not used.	Used as an optional ensemble member in Agent 2.	Studied; deferred to Capstone-II.

Table 2: Technology review: causal analysis and agent orchestration

Criterion	CausalNex (DYNOTEARS)	Tigramite (PCMCI)	DoWhy	LangGraph
Description	Python library from QuantumBlack for causal reasoning; includes NOTEARS and DYNOTEARS.	Tigramite package implementing PCMCI [12].	PyWhy library for causal effect estimation [13].	Graph-based orchestration library for LLM agents [11].
Key features	Structure learning, Bayesian networks, lagged graphs.	Lagged conditional-independence tests, nonlinear tests.	Model, identify, estimate, refute workflow.	Typed state, conditional edges, retries.
Performance	Optimisation-based; fine for tens of variables.	Heavy with nonlinear tests.	Fast for linear estimators.	Negligible overhead.
Scalability	Cubic cost in variables per iteration.	Practical for tens of variables.	Per-effect estimation.	Single process or server deployment.
Cost / licensing	Free, Apache 2.0.	Free, GPL-3.0.	Free, MIT.	Free, MIT.
Compatibility	Pins older Python and pandas versions.	Python.	Python; works with pandas.	Python and JavaScript.
Ease of use	Moderate.	Moderate; good documentation.	Clear API.	Moderate.
Security	Library only.	Library only.	Library only.	No built-in auth.
Current market	Niche, mainly consulting use.	Popular in climate science.	Widely cited in causal ML.	Fast-growing agent ecosystem.
Limitations	Dependency conflicts with the current scientific stack.	Copyleft licence; slow nonlinear tests.	Needs a causal graph as input; does not discover one.	API changes between major versions.
Relevance to project	Algorithm adopted; re-implemented in NumPy/SciPy.	Candidate for Capstone-II comparison.	Used to estimate effects on discovered edges.	Used to orchestrate the five agents.

Table 3: Technology review: storage, ingestion and transfer

Criterion	DuckDB	Fivetran	Airbyte	IBM Aspera
Description	In-process analytical SQL database [14].	Managed ELT service [15].	Open-source ELT platform [16].	IBM high-speed file transfer based on the FASP protocol [17].
Key features	Columnar, vectorised; reads CSV, Parquet, JSON, S3 directly.	Hundreds of maintained connectors, schema drift handling.	Large connector catalogue, connector builder.	UDP-based transfer, rate control, resume, encryption.
Performance	Aggregates tens of millions of rows per second on one core.	Reliable scheduled syncs.	Good; depends on connector.	Near line-rate on long-distance links.
Scalability	Larger-than-memory processing; single writer.	Fully managed.	Kubernetes deployment.	Enterprise scale.
Cost / licensing	Free, MIT.	Usage-based pricing on monthly active rows.	Open-source core; paid cloud.	Proprietary, licensed.
Compatibility	Python, R, Java, Node; no server.	Cloud warehouses as destination.	Docker or Kubernetes required.	Dedicated clients and servers.
Ease of use	Easy; standard SQL.	Very easy.	Moderate; heavy to self-host.	Moderate.
Security	File-level; no user management.	SOC 2, encryption.	Self-managed.	Encryption in transit and at rest.
Current market	Rapidly growing adoption.	Market leader in managed ELT.	Large open-source community.	Media and life-science industries.
Limitations	Single-node; one writer process.	Cost grows with volume; copies whole tables; vendor lock-in.	Several containers per sync; too heavy for small firms.	Licence cost; proprietary protocol.
Relevance to project	Used as the metric store and file engine.	Rejected on cost.	Optional route for rare systems only.	Design reference for Fast Transfer; not used.

2.3 Comparison and Analysis

The commercial BI platforms are strong at presenting data but share the same interaction model: they wait for a question. Their automated explanations are useful starting points, yet Key Influencers and Explain Data rank fields by statistical association, which cannot separate a cause from its effect or from a common driver. Root-cause tools from the AIOps field, such as CloudRanger [18], MicroCause [19] and RCD [20], do use causal graphs, but they are built for microservice telemetry (latency, error rates, CPU) and assume engineers as users. None of them produces business-facing explanations or connects to ERP tables and spreadsheets.

Causal claims require more than association [21]. Among causal discovery methods, Granger-style regression [22] is fast but tends to add edges between variables that share a driver. PCMCI handles nonlinearity at a higher computational cost and under a copyleft licence. DYNOTEARS learns contemporaneous and lagged edges together under an acyclicity constraint, which suits business metrics where effects arrive with a delay. VARLiNGAM [23] also learns lagged structure but relies on non-Gaussian noise. On the ingestion side, the choice is between paying per row (Fivetran [15]), running a heavy self-hosted platform (Airbyte [16]), or writing custom code. Transfer tools such as Aspera [17] solve the large-file problem but are proprietary. Table 4 summarises how the existing approaches cover the needs identified in Chapter 1.

Table 4: Coverage of project requirements by existing approaches

Requirement	Dashboards (Power BI, Tableau)	AIOps RCA tools	ELT platforms	KAIROS
Monitoring without a human request	Partial (alerts on thresholds)	Yes	No	Yes
Directed, lagged causal explanation	No (correlational)	Yes (IT metrics)	No	Yes (business metrics)
Plain-English report with fact checking	No	No	No	Yes
Low-cost multi-source ingestion	Built-in connectors, licensed	No	Yes, paid or heavy	Yes, free and lightweight
Large-file transfer strategy	No	No	Bulk copy	Push-down, edge, Fast Transfer

2.4 Review Summary

Three findings stand out. Monitoring, causal reasoning and narrative explanation exist separately in different tools, but no reviewed product combines them for business metrics. The

open-source building blocks are mature and free: Prophet and robust statistics for detection, DYNOTEARS and DoWhy for causality, LangGraph for orchestration and DuckDB for storage. The cost of data movement is driven mainly by how much data is copied, not by the protocol, so a design that moves aggregated metrics instead of raw rows can avoid both licence fees and long transfers. These findings directly shaped the design described in Chapter 3.

2.5 Research Gap

The gaps identified in the review are grouped by category in Table 5.

Table 5: Research gaps identified from the technology review

Category	Gap in existing systems	Evidence	KAIROS response
Functional	BI tools do not monitor on their own initiative	Power BI and Tableau require a user session	Scheduled five-agent pipeline (ProBI)
Technical	Explanations are correlational, not causal or lagged	Key Influencers, Explain Data	DYNOTEARS causal graph (CBMG) with lag-aware ranking
Reliability	LLM narratives can state wrong numbers	Hallucination survey [9]	LLM-Modulo verifier on every report
Usability	Each new source needs manual schema mapping	ELT setup and semantic layers are hand-modelled	SLAC infers timestamp, shape and metrics
Cost	Connectors are priced by volume or are heavy to host	Fivetran pricing, Airbyte deployment	Five protocol adapters; aggregation at the source
Performance	Single-stream FTP is slow for large files	TCP throughput limits [24]	Edge aggregation and parallel, resumable Fast Transfer

3. Objectives and Methodology

3.1 Project Objectives

Five SMART objectives were set at the start of the semester and revised after the mid-term review, when the ingestion questions were raised. They are listed in Table 6.

Table 6: SMART objectives of the project

No.	Objective	Measure of success	Deadline
O1	Build a five-agent pipeline that runs every 15 minutes without human initiation and stores data idempotently	End-to-end run under 30 s for 15 metrics; no duplicate rows on re-runs	Week 8
O2	Detect injected anomalies in seasonal business metrics	Event recall of at least 95% and point precision of at least 70%	Week 10
O3	Identify the causal origin of an incident	Causal F1 of at least 0.85; origin correct in at least 90% of runs	Week 10
O4	Generate executive reports whose numbers are verified before delivery	At least 95% of numeric misstatements caught	Week 11
O5	Ingest from at least four source types at low cost and handle large files	Incremental sync; a 100 MB export moved at least 3 times faster than single-stream FTP	Week 11

3.2 Proposed Approach

Development followed two-week Agile sprints with a working build at the end of each sprint. The first sprints produced the five agents and the dashboard (mid-term prototype). After the mid-term review, the work focused on the ingestion layer, the causal engine and verification. Quality was maintained through automated tests that run on every change, code review within the team, and a benchmark script that regenerates every reported number from scratch, so results can be checked by anyone with the repository.

3.3 System Architecture

The overall architecture is shown in Figure 1. Data from five kinds of sources passes through an ingestion layer into a DuckDB store. Five agents then run in sequence under a LangGraph state machine, triggered every 15 minutes by a background scheduler. A FastAPI [25] service exposes every function, and a React dashboard sits on top of it.

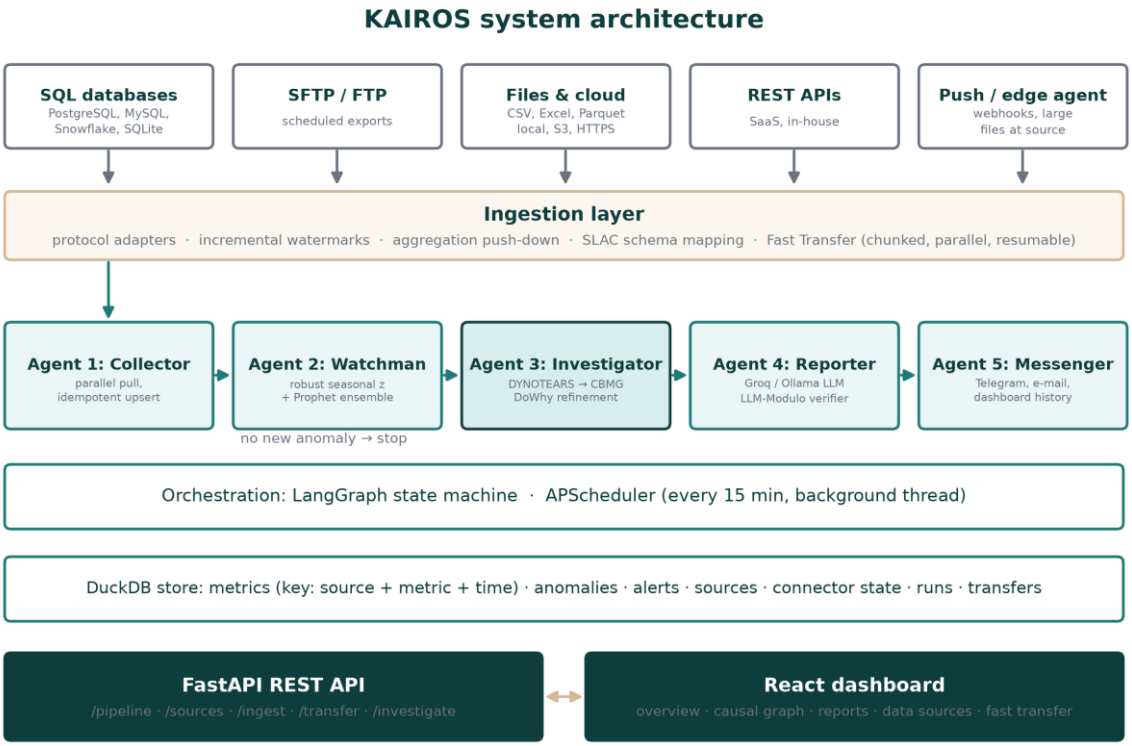


Figure 1: KAIROS system architecture

The control flow of one monitoring cycle appears in Figure 2. The graph stops after detection when no new alert-worthy anomaly is found, so a quiet cycle costs a fraction of a second and no alert is repeated.



Figure 2: LangGraph state machine for one monitoring cycle

3.3.1 Agent 1: Data Collector

Every source implements one contract, `fetch(state)`, which returns rows in a canonical form (metric name, value, timestamp) together with an updated state. The state holds a watermark or the list of files already processed, so each cycle reads only new data. Rows are written with an upsert on the key (source, metric, timestamp), so a repeated sync updates values instead of duplicating them, a property that makes retries safe in any data pipeline [26]. Sources run in parallel with a timeout each, so one slow system cannot delay the others.

3.3.2 Agent 2: Anomaly Watchman

Each series is compared with a seasonal baseline (the median value for the same hour of day). The residual is scaled by the median absolute deviation, giving a robust z-score $z = (x - m) / (1.4826 \times \text{MAD})$, which is not distorted by the anomalies it is looking for [4]. Points with $|z| \geq 3.5$ are flagged, and severity is graded at 3.5, 5, 7 and 10 robust standard deviations. Prophet [3] can be enabled as a second detector. Every anomaly is stored, so it is reported as new exactly once.

3.3.3 Agent 3: Causal Investigator

The metrics are aligned on a common time grid, winsorised so that the anomalies do not distort the learned structure, and standardised. DYNOTEARS [7] then fits the structural vector autoregression $X_t = X_t W + X_{(t-1)} A + e$, where W holds same-step effects and A holds one-step lagged effects. It minimises the squared residual plus L1 penalties, subject to the NOTEARS acyclicity constraint $h(W) = \text{tr}(e^{(W \circ W)}) - d = 0$ [6], using an augmented Lagrangian with L-BFGS-B. Edges with weight of at least 0.15 form the Causal Business Metric Graph (CBMG). Edge confidence is the share of bootstrap refits that keep the edge. For each anomalous metric, the agent walks back through its ancestors, scores each chain by strength, confidence and whether the cause itself was anomalous shortly before, and marks as origin any anomalous cause with no anomalous upstream cause. DoWhy [13] estimates the size of each direct effect.

3.3.4 Agent 4: NL Reporter

A prompt containing only computed facts is sent to an LLM (Groq first, local Ollama as fallback). Following the LLM-Modulo pattern [27], the draft is then checked by a deterministic verifier: every number must match a computed fact at the precision written, and every metric name must exist. A failed draft is returned once with the list of errors; if it fails again, a template report that is correct by construction is used.

3.3.5 Agent 5: Messenger

The verified report is sent through the Telegram Bot API [28] and e-mail and stored in an alert history shown on the dashboard. A cooldown prevents repeated alerts for the same metric.

3.3.6 Ingestion Strategy for Any Source

The ingestion design answers the review question directly. Instead of one connector per product, KAIROS implements five protocol adapters that together cover almost any system.

The principle is to move the computation rather than the data: for a large table, the aggregation query runs inside the source database; for large raw files, a small edge agent aggregates them next to the data and pushes only KPIs; for small scheduled exports, SFTP/FTP [29] fetches only files not seen before. Figure 3 shows how a source is routed. The Semantic Layer Auto-Construction (SLAC) module inspects a sample of any new source and proposes the mapping (timestamp column, long or wide shape, metric columns, and whether raw records need aggregation), which removes most manual setup.

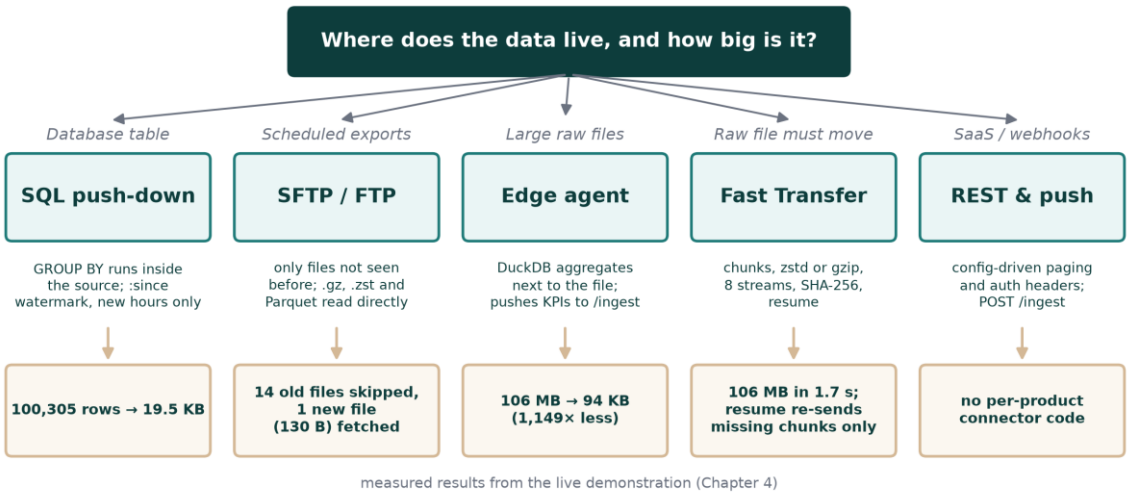


Figure 3: Ingestion routes by data location and size, with measured results

When a raw file must move, the Fast Transfer service is used (Figure 4). The file is split into chunks, each chunk is compressed (gzip or Zstandard [30]) and hashed, and several chunks are sent in parallel. A single TCP stream is limited by window size and round-trip time [24], so parallel streams use more of the link [31]. If the connection drops, the client asks which chunks are missing and sends only those. The server verifies each chunk, writes it in place, and re-hashes the whole file before accepting it.

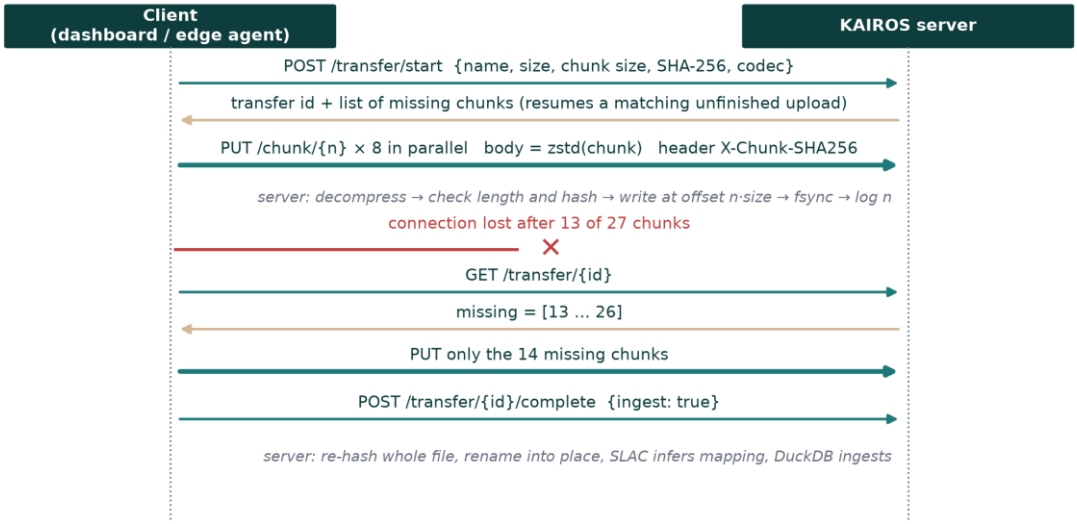


Figure 4: Fast Transfer protocol, including resume after a dropped connection

3.4 Tools and Technologies

The tools selected after the review are listed in Table 7. All of them are free and open source.

Table 7: Tools and technologies used

Layer	Tools
Languages	Python 3.13, JavaScript (ES2022), SQL
Backend and orchestration	FastAPI, Uvicorn, LangGraph, APScheduler, Pydantic
Data and storage	DuckDB, pandas, PyArrow, SQLAlchemy
Detection and causality	NumPy, SciPy (DYNOTEARS), Prophet, DoWhy, causal-learn (VARLiNGAM baseline), NetworkX
LLM and delivery	Groq API, Ollama (Llama 3.1 8B), Telegram Bot API, Gmail SMTP
Connectors and transfer	paramiko (SFTP), ftplib (FTP), httpx (REST), gspread (Sheets), zstandard, gzip
Frontend	React 18, Vite, Tailwind CSS, Recharts
Testing and tooling	pytest, pyftpdlib (test FTP server), Docker, Git, VS Code

3.5 Implementation Plan

The semester plan, with milestones at the three evaluations, is shown in Figure 5. Work on agents and the dashboard ran in parallel among the three team members, and the ingestion and verification work was added after Evaluation 2 in response to the review.

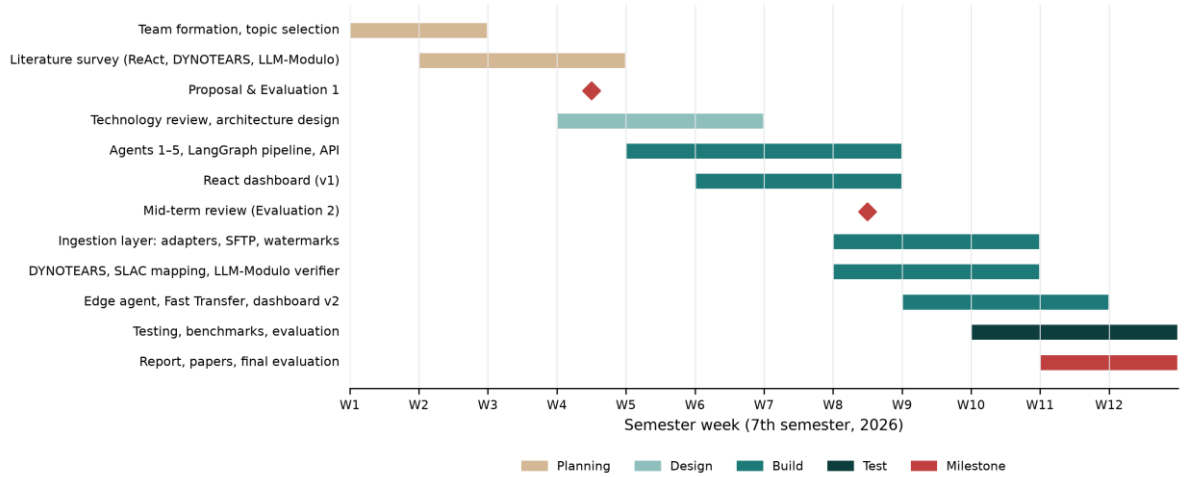


Figure 5: Project timeline (Gantt chart) for the 7th semester

3.6 Novelty

The project contributes four elements that were not found together in any reviewed system.

- **Proactive BI (ProBI):** a complete loop from ingestion to verified alert that runs on a schedule, stores every anomaly, and alerts only on first sightings.
- **Causal Business Metric Graph (CBMG):** DYNOTEARS applied to business KPIs, with a ranking that combines edge strength, bootstrap confidence and the timing of anomalies to name the origin of an incident rather than a list of correlated metrics.
- **Verified narration:** an LLM-Modulo critic that checks numbers at the precision written, so a fluent but wrong report never reaches a manager.
- **Low-cost ingestion for any source:** protocol-level adapters, SLAC mapping, aggregation push-down, an edge agent and a resumable parallel transfer service, which together answer the cost and large-file concerns raised in the review.

The evidence for each claim is presented in Chapter 4.

4. Implementation and Results

4.1 Implementation Details

The backend consists of about 5,100 lines of Python and the dashboard of about 1,500 lines of JavaScript. Implementation proceeded in five steps: the DuckDB schema with primary keys and migrations; the connector framework and the six adapters; the detection and causal models, including a NumPy/SciPy implementation of DYNOTEARS written because the existing library did not install on Python 3.13; the report verifier; and finally the transfer service, the edge agent and the dashboard. Every number in this chapter can be regenerated with two scripts in the repository, `benchmark/evaluate_models.py` and `benchmark/benchmark_ingestion.py`.

4.2 System Features

The dashboard has five pages. The Overview page (Figure 6) shows severity counts, a filterable anomaly table and the time series of the selected metric with anomalies marked by severity.

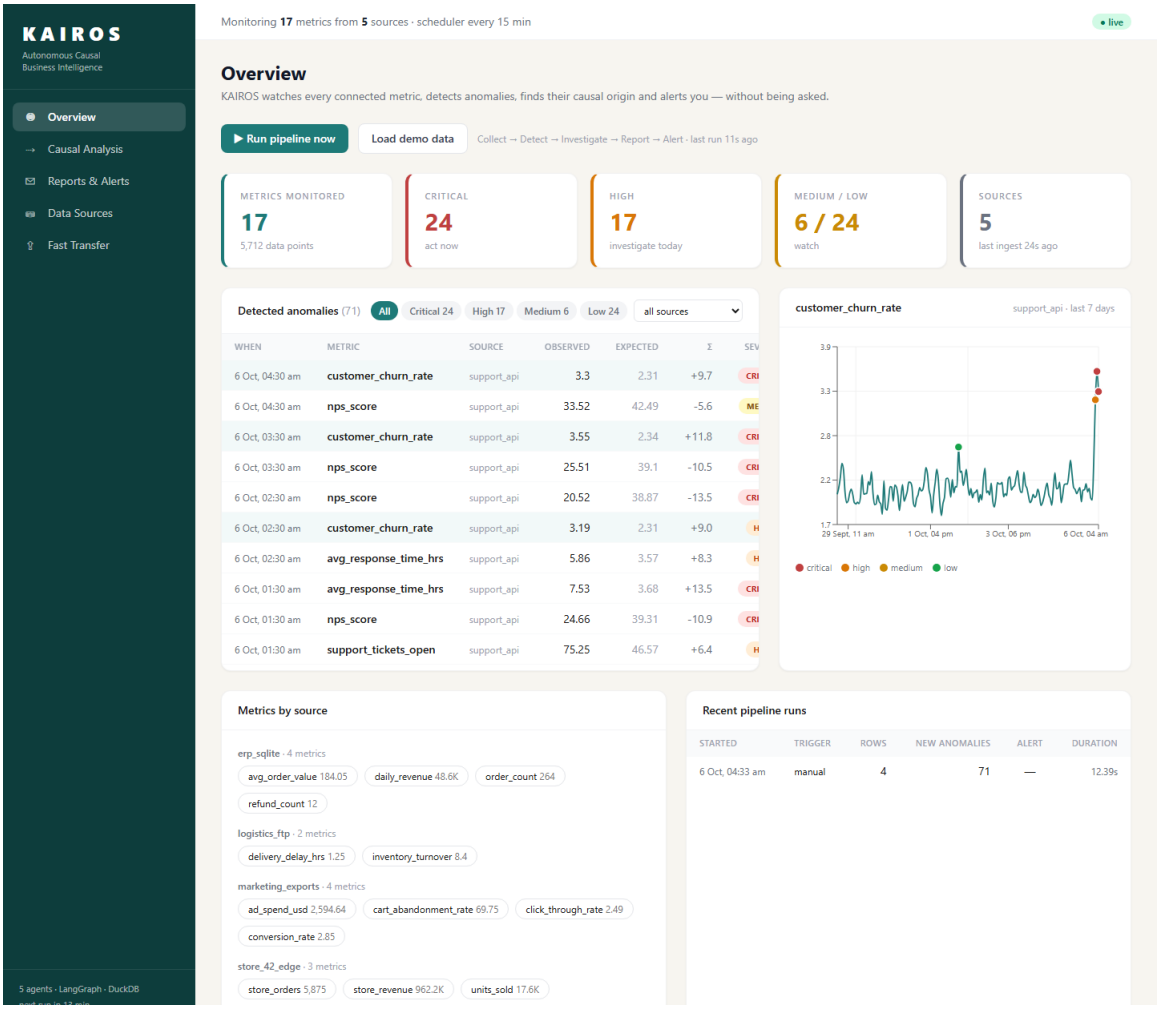


Figure 6: Overview page after a monitoring cycle on live demo sources

The Data Sources page (Figure 7) lists every connected system with the rows and bytes moved in the last sync. New sources are added through a form; the Test button shows a sample and the mapping proposed by SLAC.

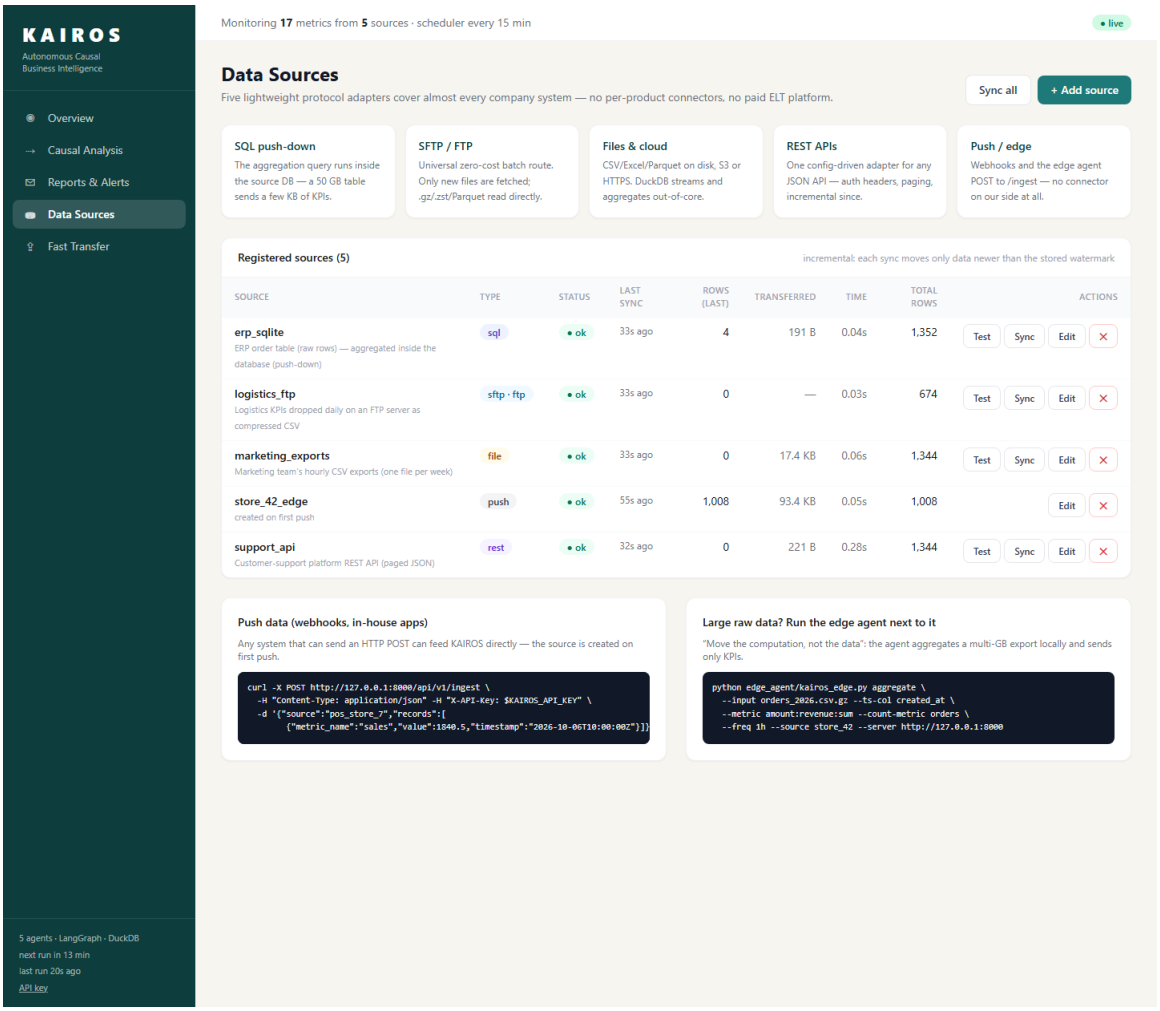


Figure 7: Data Sources page with SQL, FTP, file, REST and push sources

The Fast Transfer page (Figure 8) uploads files of any size from the browser with live chunk progress, pause and resume, and shows the measured benchmark.

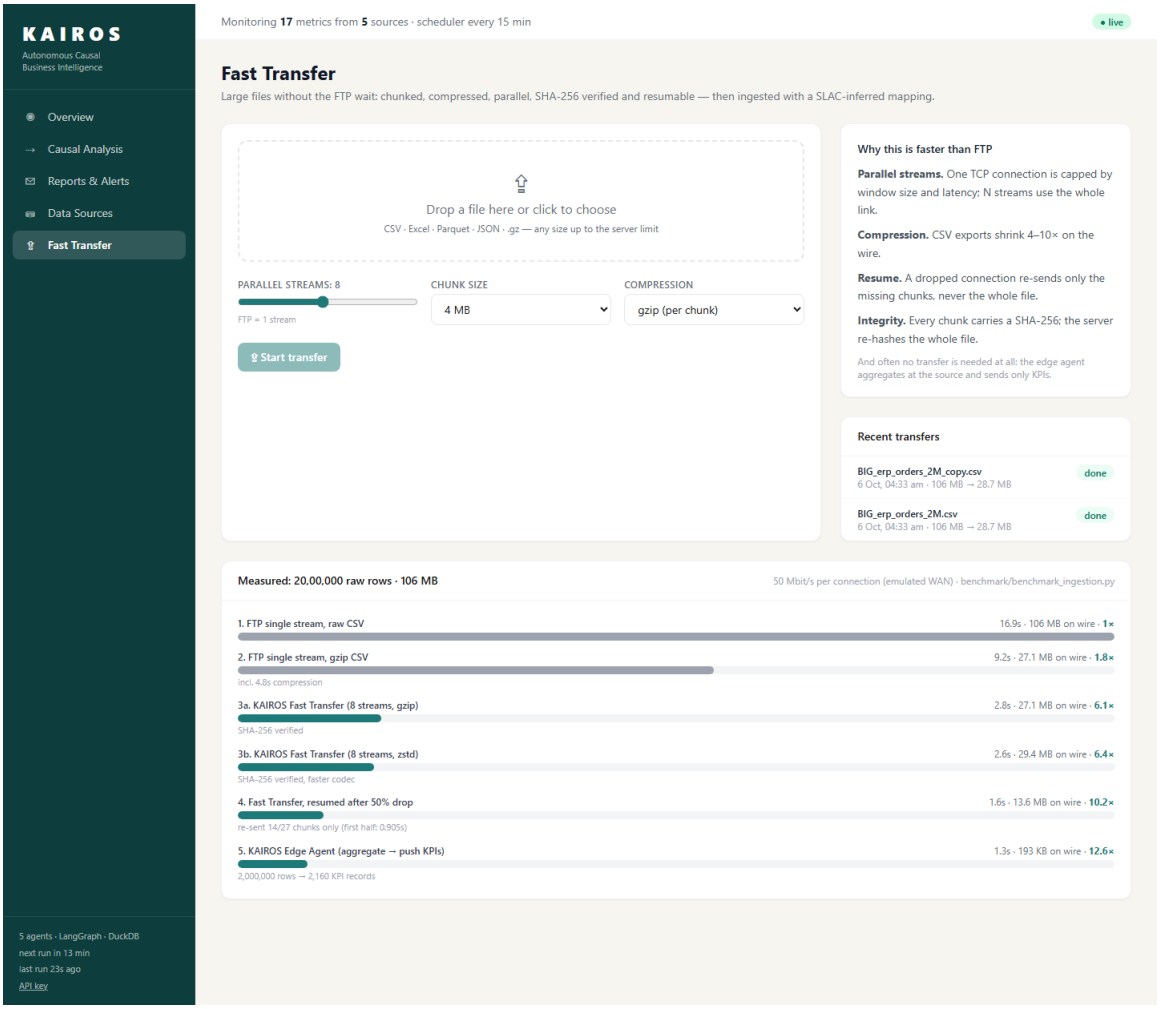


Figure 8: Fast Transfer page with transfer history and benchmark results

4.3 Testing and Validation

Testing combined automated tests, a live demonstration with simulated company systems, and two benchmark scripts. The automated suite is summarised in Table 8; all 36 tests pass. The connector tests use a real FTP server and a real HTTP server started inside the test, not mocks.

Table 8: Automated test suite

Area	Tests	What is verified
Connectors	14	Long and wide mapping, CSV/gzip/Parquet/Excel, SQL push-down, REST paging, FTP new-file tracking, failure isolation
Core models	10	Upsert idempotency, detection, DYNOTEARS recovery, SLAC inference, verifier, alert gating
Fast Transfer	5	Chunk hashing, resume, corrupt-chunk rejection, whole-file checksum
API end to end	7	Seed, pipeline, no duplicate alerts, sources, push, upload, transfer, API key
Total	36	All passing

4.4 Results and Analysis

4.4.1 Multi-Source Ingestion

Four simulated company systems were connected: an ERP database with 100,305 raw order rows, a folder of weekly marketing CSV exports, an FTP server receiving daily compressed logistics files, and a paged REST API from a support platform. Table 9 records what happened in the live demonstration.

Table 9: Live ingestion demonstration

Step	Result
First sync of all four sources	4,704 KPI rows collected; ERP sent 19.5 KB for 100,305 raw rows because the GROUP BY ran inside the database
Second sync	No new rows from FTP, files or REST; nothing duplicated
New file dropped on the FTP server	Only that file (130 bytes) downloaded; the 14 earlier files were skipped
Edge agent on a 2-million-row export (105.6 MB)	Aggregated locally in 0.6 s; 94.1 KB pushed, 1,149 times less data
Fast Transfer of the same file	Sent as 28.7 MB in 1.7 s with SHA-256 verified on the server
Connection cut after 13 of 27 chunks	Resume re-sent only the 14 missing chunks

4.4.2 Large-File Transfer Benchmark

To answer the reviewers' concern about FTP, the same 2-million-row export (105.6 MB) was moved by each method between two processes on one machine, with every connection capped at 50 Mbit/s to emulate a wide-area link. The cap was applied in the same way to FTP and to Fast Transfer. Results are in Table 10 and Figure 9.

Table 10: Ingestion benchmark for a 105.6 MB raw export

Method	Time (s)	Bytes sent	Speed-up
FTP, single stream, raw CSV	16.90	105.6 MB	1.0×
FTP, single stream, gzip (incl. 4.8 s compression)	9.18	27.1 MB	1.8×
Fast Transfer, 8 streams, gzip	2.76	27.1 MB	6.1×
Fast Transfer, 8 streams, zstd	2.63	29.4 MB	6.4×
Fast Transfer, resumed after 50% drop	1.65	13.6 MB	10.2×
Edge agent (aggregate, push KPIs)	1.34	192.7 KB	12.6×

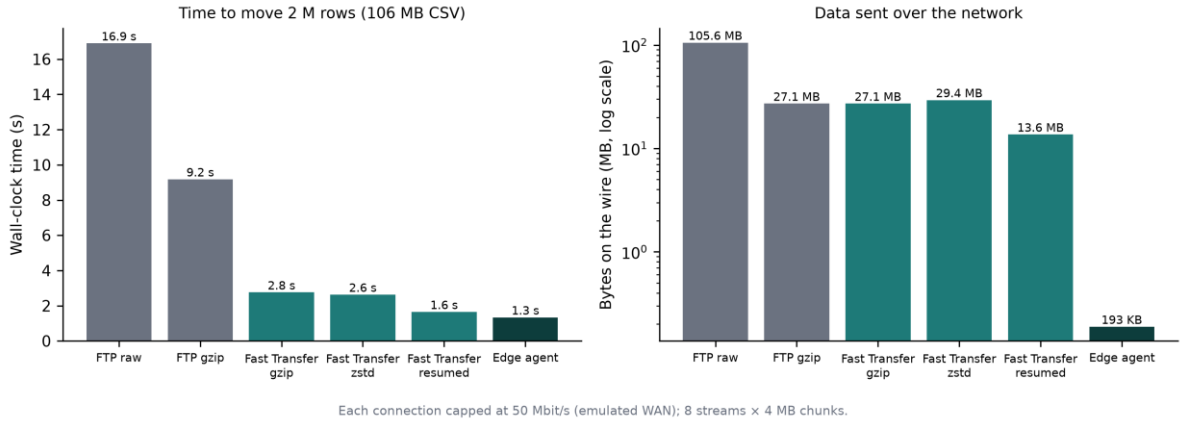


Figure 9: Transfer time and bytes on the wire for each method

Fast Transfer was 6.4 times faster than plain FTP while sending about a quarter of the bytes, and a broken transfer cost only the missing half. Edge aggregation was faster still and reduced network traffic by a factor of about 550, which supports the central argument that most large-data cases should not transfer raw data at all.

4.4.3 Anomaly Detection

Detection was evaluated over ten random seeds of the synthetic benchmark (15 metrics, 336 hourly points, two cascading incidents and six isolated anomalies). An event counts as detected if any point of the affected metric is flagged inside the event window. Table 11 gives the results and Figure 10 shows typical output.

Table 11: Anomaly detection over ten seeds (mean ± standard deviation)

Detector	Event recall	Point precision	Time per cycle (s)
Robust seasonal z-score	100.0 ± 0.0%	77.2 ± 6.2%	0.23
Robust z-score + Prophet ensemble (5 seeds)	100.0 ± 0.0%	75.2 ± 4.1%	3.34

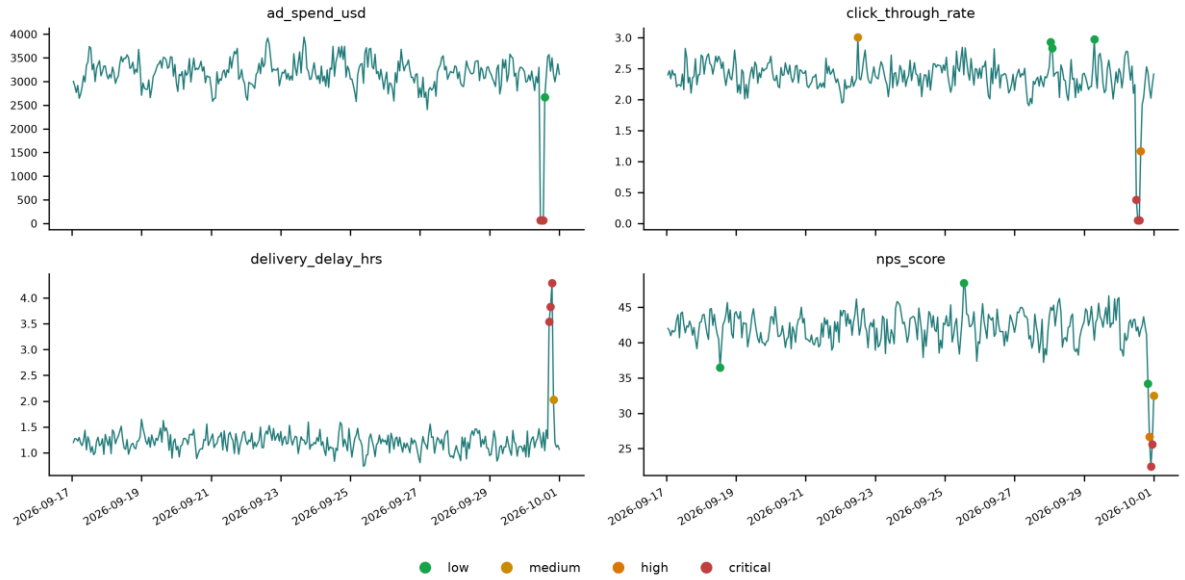


Figure 10: Detected anomalies on four metrics, coloured by severity

Every injected event was found. The flags counted as false positives are mostly points just outside the event windows, where the cascade is still settling. Prophet did not improve precision on this benchmark and was about fourteen times slower, so it is kept as an option rather than the default.

4.4.4 Causal Discovery and Root Cause

The learned graphs were compared with the twelve true causal edges of the benchmark. Table 12 and Figure 12 compare DYNOTEARS with three baselines.

Table 12: Causal discovery against ground truth over ten seeds

Method	Precision	Recall	F1	SHD
DYNOTEARS (KAIROS)	1.00 ± 0.00	0.95 ± 0.04	0.97 ± 0.02	0.6 ± 0.5
Granger (lagged OLS)	0.83 ± 0.13	0.99 ± 0.03	0.90 ± 0.08	2.8 ± 2.5
VARLiNGAM	0.78 ± 0.11	0.99 ± 0.03	0.87 ± 0.08	3.7 ± 2.5
Pearson correlation	0.44 ± 0.01	0.58 ± 0.07	0.50 ± 0.03	7.0 ± 0.9

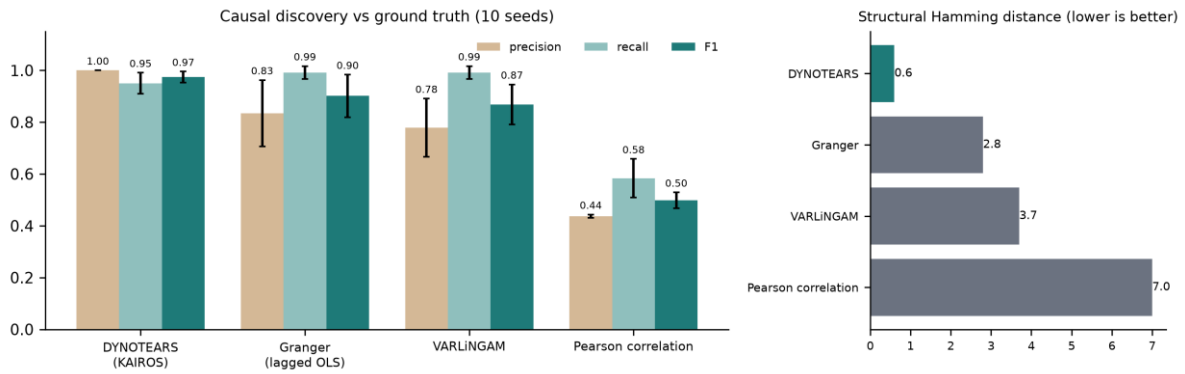


Figure 11: Precision, recall, F1 and structural Hamming distance of each method

DYNOTEARS never added a false edge in ten runs; the occasional miss was the weakest true link (average order value to revenue). The baselines found almost every true edge but added spurious ones, and correlation, which has no direction, performed worst. Figure 11 shows a learned graph next to the ground truth. In the full pipeline, KAIROS named both injected origins, advertising spend and delivery delay, in all ten runs, and the investigation took 1.6 s on average.

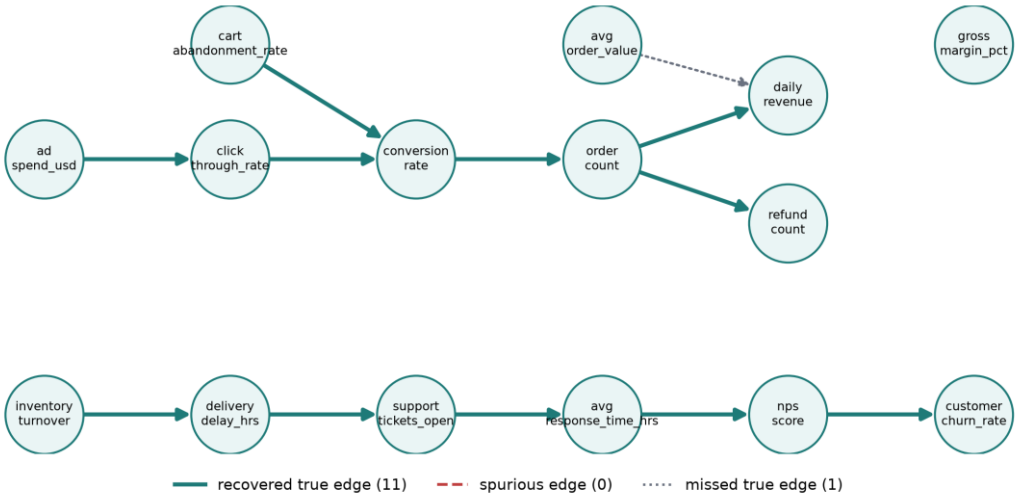


Figure 12: Graph learned by DYNOTEARS (seed 3) compared with the true structure

The same analysis on the dashboard is shown in Figure 13. Each chain lists strength, bootstrap confidence, lag and whether the cause moved before the effect.

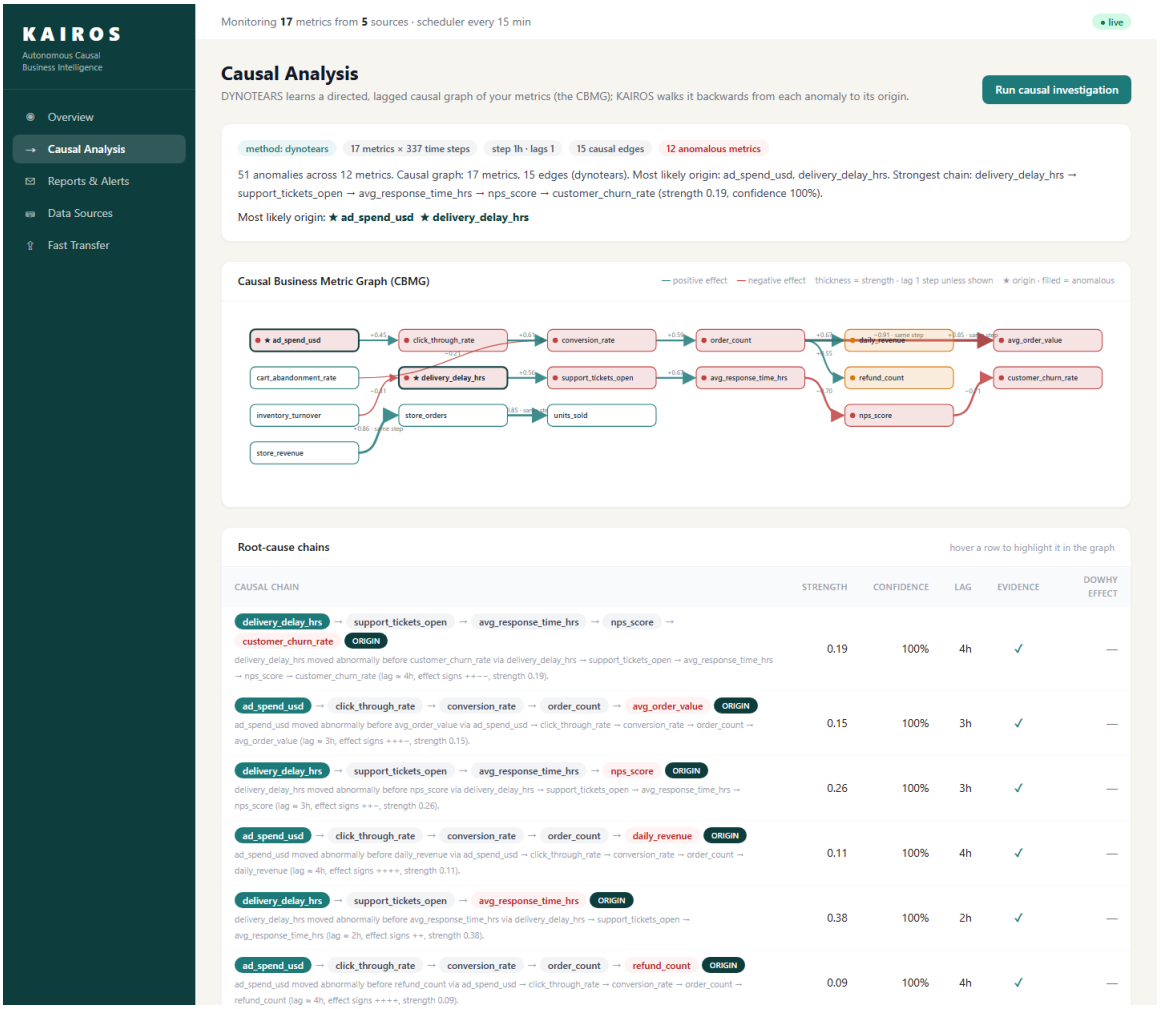


Figure 13: Causal Analysis page with the CBMG and ranked root-cause chains

4.4.5 Report Verification

The verifier was tested by corrupting numbers and metric names in correct reports. Results are in Table 13. An earlier version accepted 7.03 as a match for 6.71 because both round to 7; comparing at the written precision raised the catch rate from 61% to 98.5% without rejecting faithful rounding.

Table 13: LLM-Modulo verifier stress test

Test	Cases	Result
Faithful reports accepted	5	100%
Corrupted numbers caught	200	98.5%
Invented metric names caught	50	100%
Faithful one-decimal rounding accepted	200	100%

In the live run, the Groq model's report passed verification on the first attempt (Figure 14). The full five-agent cycle on live sources took 12.4 s.

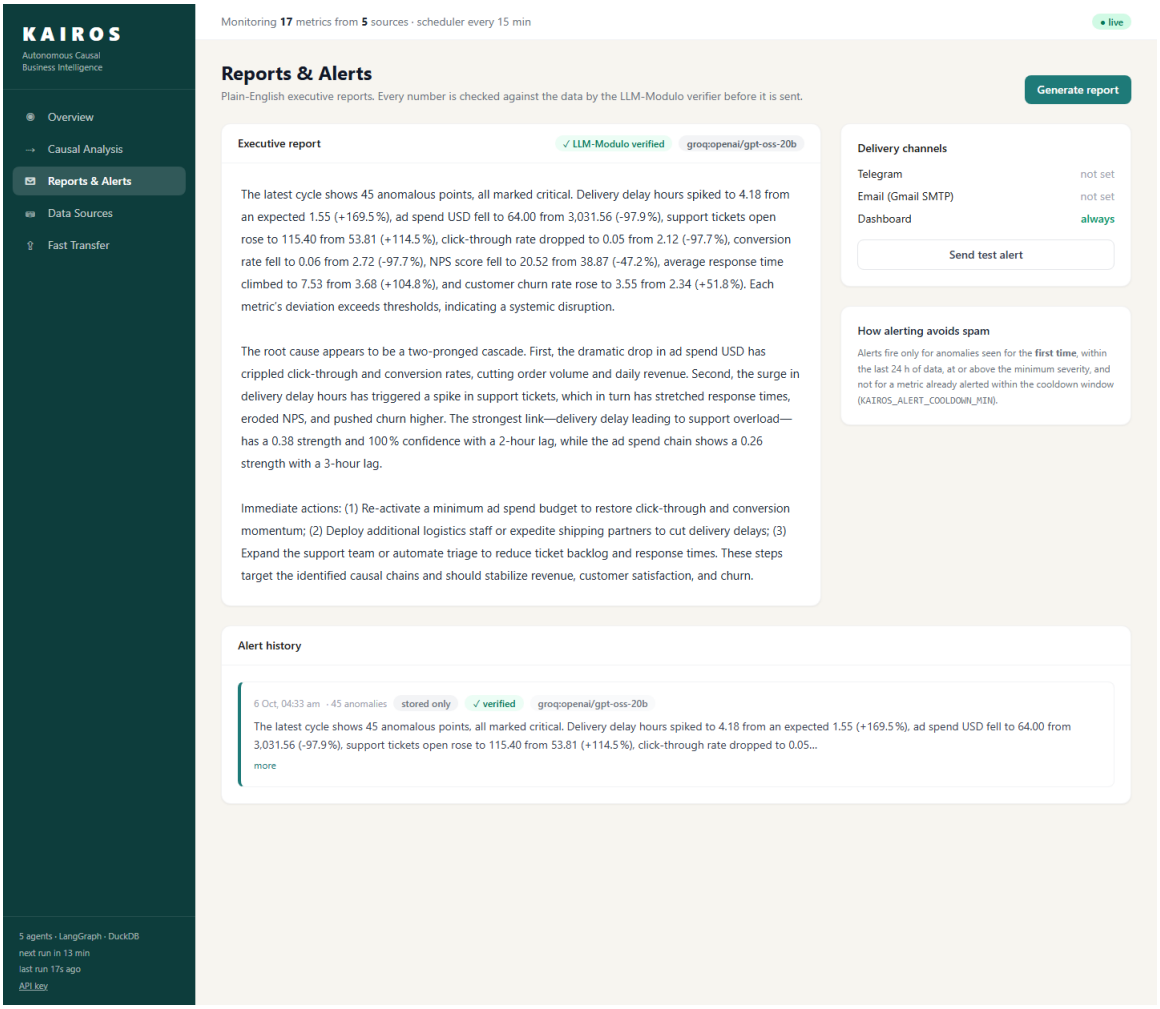


Figure 14: Reports page with an LLM-written report marked as verified

4.5 Discussion

The results support the two main claims. First, causal discovery gives better explanations than the correlation logic behind current BI features: on data with a known structure, it identified the origin of every incident, which is exactly what an analyst would otherwise trace by hand. Second, the cost and speed of ingestion depend mostly on what is moved. Aggregating at the source turned the reviewers' large-file problem into a transfer of a few hundred kilobytes, and Fast Transfer covers the cases where the raw file is genuinely needed. Two cautions apply. The benchmark data follows a linear model with known lags, which favours DYNOTEARS, and the transfer link was emulated on one machine. Both points are treated as limitations in Chapter 5.

5. Conclusion and Future Work

5.1 Summary

KAIROS shows that business monitoring can move from a pull model, where someone has to ask, to a push model in which the system finds the problem, explains its cause and reports it in verified language. The project delivered a working five-agent system, a dashboard, an edge agent, a transfer service, 36 automated tests and two reproducible benchmarks.

5.2 Objectives Achievement

All five objectives were met, as summarised in Table 14.

Table 14: Achievement of the SMART objectives

Objective	Target	Achieved
O1 Autonomous pipeline	< 30 s per cycle, no duplicates	12.4 s; idempotent upsert verified by tests
O2 Anomaly detection	≥ 95% recall, ≥ 70% precision	100% recall, 77.2% precision
O3 Causal origin	F1 ≥ 0.85, origin ≥ 90%	F1 0.97, origin 100%
O4 Verified reports	≥ 95% misstatements caught	98.5% caught, 100% faithful accepted
O5 Low-cost ingestion	≥ 4 source types, ≥ 3× faster than FTP	5 adapter types, 6.4× (edge 12.6×)

5.3 Limitations

- Evaluation used synthetic data whose linear, lagged structure matches the assumptions of DYNOTEARS. Results on real company data may be lower.
- DYNOTEARS assumes no hidden common causes and linear effects; a driver that KAIROS does not monitor can appear as a spurious link.
- The transfer benchmark ran on one machine with an emulated 50 Mbit/s link. Real WAN tests remain to be done.
- The verifier checks numbers and metric names, not qualitative claims. In one live report the LLM described all anomalies as critical, which was not fully correct and was not caught.
- The Anomaly Transformer was reviewed but not implemented, and no user study with business staff has been carried out.

5.4 Lessons Learned

The most useful lesson came from the review itself: a question about FTP led to the realisation that the size of the data to be moved matters more than the protocol. Technically, writing

DYNOTEARS directly in NumPy and SciPy took longer than installing a library but removed a dependency conflict and made the method easier to explain. Testing against a real FTP server instead of mocks caught two bugs in file tracking. Keeping every number reproducible through scripts made the report faster to write and easier to defend. Within the team, splitting work by agent and meeting at sprint boundaries worked well; integration should have started earlier.

5.5 Recommendations

Before production use, KAIROS should be run in shadow mode on one real business unit for several weeks, with analysts marking each alert as useful or not. SFTP or FTPS should always be used instead of plain FTP, with read-only accounts.

5.6 Future Scope for Capstone Project-II

- Evaluate on public datasets (Olist e-commerce, Superstore) and on data from a partner company.
- Add nonlinear causal discovery (PCMCI) and test robustness to hidden confounders.
- Implement the Anomaly Transformer and compare it with the current detector.
- Extend the verifier to qualitative claims such as severity and direction of change.
- Measure Fast Transfer over real long-distance links, and add change data capture for databases [32].
- Add multi-tenant deployment, role-based access and a user study with managers.

In the longer term, KAIROS could become a low-cost monitoring service for small businesses that cannot afford a data team, delivering the kind of root-cause analysis that today requires an analyst.

Acknowledgment

The team is grateful to Dr. K RajKumar for steady guidance throughout the semester, and for the questions during reviews that shaped the most important parts of this work. Thanks are due to the evaluation panel of the second review, whose comments on data ingestion and large-file transfer led directly to the ingestion layer and the Fast Transfer service. The School of Technology, Woxsen University, provided the computing resources, library access and the structure of the Capstone programme. The project depends on the work of many open-source communities, in particular the developers of DuckDB, FastAPI, LangGraph, Prophet, DoWhy, causal-learn, React and the authors of the DYNOTEARS and LLM-Modulo papers. Classmates who tested early versions of the dashboard gave useful feedback on its design. Finally, the team thanks their families for their patience and support during the long evenings this project required.

References

[1] J. Xu, H. Wu, J. Wang, and M. Long, "Anomaly Transformer: Time series anomaly detection with association discrepancy," in *Proc. Int. Conf. Learning Representations (ICLR)*, 2022.

[2] "Power BI documentation: Key influencers visual," Microsoft. [Online]. Available: https://learn.microsoft.com/power-bi/visuals/power-bi-visualization-influencers. [Accessed: Oct. 6, 2026].

[3] S. J. Taylor and B. Letham, "Forecasting at scale," *The American Statistician*, vol. 72, no. 1, pp. 37–45, 2018.

[4] C. Leys, C. Ley, O. Klein, P. Bernard, and L. Licata, "Detecting outliers: Do not use standard deviation around the mean, use absolute deviation around the median," *Journal of Experimental Social Psychology*, vol. 49, no. 4, pp. 764–766, 2013.

[5] V. Chandola, A. Banerjee, and V. Kumar, "Anomaly detection: A survey," *ACM Computing Surveys*, vol. 41, no. 3, pp. 1–58, 2009.

[6] X. Zheng, B. Aragam, P. Ravikumar, and E. P. Xing, "DAGs with NO TEARS: Continuous optimization for structure learning," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 31, 2018.

[7] R. Pamfil, N. Sriwattanaworachai, S. Desai, P. Pilgerstorfer, K. Georgatzis, P. Beaumont, and B. Aragam, "DYNOTEARS: Structure learning from time-series data," in *Proc. 23rd Int. Conf. Artificial Intelligence and Statistics (AISTATS)*, PMLR vol. 108, 2020, pp. 1595–1605.

[8] T. Brown *et al.*, "Language models are few-shot learners," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, 2020, pp. 1877–1901.

[9] Z. Ji, N. Lee, R. Frieske, T. Yu, D. Su, Y. Xu, E. Ishii, Y. Bang, A. Madotto, and P. Fung, "Survey of hallucination in natural language generation," *ACM Computing Surveys*, vol. 55, no. 12, pp. 1–38, 2023.

[10] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao, "ReAct: Synergizing reasoning and acting in language models," in *Proc. Int. Conf. Learning Representations (ICLR)*, Kigali, Rwanda, 2023.

[11] "LangGraph documentation," LangChain, Inc. [Online]. Available: https://langchain-ai.github.io/langgraph/. [Accessed: Oct. 6, 2026].

[12] J. Runge, P. Nowack, M. Kretschmer, S. Flaxman, and D. Sejdinovic, "Detecting and quantifying causal associations in large nonlinear time series datasets," *Science Advances*, vol. 5, no. 11, eaau4996, 2019.

[13] A. Sharma and E. Kiciman, "DoWhy: An end-to-end library for causal inference," arXiv:2011.04216, 2020.

[14] M. Raasveldt and H. Mühleisen, "DuckDB: An embeddable analytical database," in *Proc. ACM SIGMOD Int. Conf. Management of Data*, Amsterdam, Netherlands, 2019, pp. 1981–1984.

[15] "Fivetran pricing: Monthly active rows," Fivetran. [Online]. Available: https://fivetran.com/docs/usage-based-pricing. [Accessed: Oct. 6, 2026].

[16] "Airbyte documentation," Airbyte, Inc. [Online]. Available: https://docs.airbyte.com/. [Accessed: Oct. 6, 2026].

[17] "IBM Aspera FASP high-speed transport," IBM. [Online]. Available: https://www.ibm.com/products/aspera. [Accessed: Oct. 6, 2026].

[18] P. Wang, J. Xu, M. Ma, W. Lin, D. Pan, Y. Wang, and P. Chen, "CloudRanger: Root cause identification for cloud native systems," in *Proc. 18th IEEE/ACM Int. Symp. Cluster, Cloud and Grid Computing (CCGRID)*, 2018, pp. 492–502.

[19] Y. Meng, S. Zhang, Y. Sun, R. Zhang, Z. Hu, Y. Zhang, C. Jia, Z. Wang, and D. Pei, "Localizing failure root causes in a microservice through causality inference," in *Proc. IEEE/ACM 28th Int. Symp. Quality of Service (IWQoS)*, 2020.

[20] M. A. Ikram, S. Chakraborty, S. Mitra, S. Saini, S. Bagchi, and M. Kocaoglu, "Root cause analysis of failures in microservices through causal discovery," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 35, 2022.

[21] J. Pearl, *Causality: Models, Reasoning, and Inference*, 2nd ed. Cambridge, U.K.: Cambridge Univ. Press, 2009.

[22] C. W. J. Granger, "Investigating causal relations by econometric models and cross-spectral methods," *Econometrica*, vol. 37, no. 3, pp. 424–438, 1969.

[23] A. Hyvärinen, K. Zhang, S. Shimizu, and P. O. Hoyer, "Estimation of a structural vector autoregression model using non-Gaussianity," *Journal of Machine Learning Research*, vol. 11, pp. 1709–1731, 2010.

[24] M. Mathis, J. Semke, J. Mahdavi, and T. Ott, "The macroscopic behavior of the TCP congestion avoidance algorithm," *ACM SIGCOMM Computer Communication Review*, vol. 27, no. 3, pp. 67–82, 1997.

[25] "FastAPI documentation," S. Ramírez. [Online]. Available: https://fastapi.tiangolo.com/. [Accessed: Oct. 6, 2026].

[26] M. Kleppmann, *Designing Data-Intensive Applications*. Sebastopol, CA, USA: O'Reilly Media, 2017.

[27] S. Kambhampati, K. Valmeekam, L. Guan, M. Verma, K. Stechly, S. Bhambri, L. Saldyt, and A. Murthy, "Position: LLMs can't plan, but can help planning in LLM-Modulo frameworks," in *Proc. 41st Int. Conf. Machine Learning (ICML)*, PMLR vol. 235, 2024.

[28] "Telegram Bot API," Telegram. [Online]. Available: https://core.telegram.org/bots/api. [Accessed: Oct. 6, 2026].

[29] J. Postel and J. Reynolds, "File Transfer Protocol (FTP)," IETF RFC 959, Oct. 1985.

[30] Y. Collet and M. Kucherawy, "Zstandard compression and the application/zstd media type," IETF RFC 8878, Feb. 2021.

[31] T. J. Hacker, B. D. Athey, and B. Noble, "The end-to-end performance effects of parallel TCP sockets on a lossy wide-area network," in *Proc. 16th Int. Parallel and Distributed Processing Symp. (IPDPS)*, 2002.

[32] "Debezium documentation," Red Hat. [Online]. Available: https://debezium.io/documentation/. [Accessed: Oct. 6, 2026].

Appendix A: Source Code Snippets

A.1 DYNOTEARS objective and gradient (`backend/models/dynotears.py`). The weights are split into positive and negative parts so that the L1 penalty becomes linear and L-BFGS-B can be used with simple bounds.

```
def objective(x, rho, alpha):
    W, A = unpack(x)
    R    = Xt - Xt @ W - Xlags @ A                    # SVAR residual
    loss = 0.5 / n * np.sum(R ** 2)
    h, E = h_func(W)                                  # h(W) = tr(exp(W*W)) - d
    obj  = loss + 0.5 * rho * h * h + alpha * h \
           + lambda_w * x[:2*d*d].sum() + lambda_a * x[2*d*d:].sum()
    g_w  = -1.0 / n * Xt.T @ R + (rho * h + alpha) * (E.T * 2 * W)
    g_a  = -1.0 / n * Xlags.T @ R
    grad = np.concatenate([(g_w + lambda_w).ravel(), (-g_w + lambda_w).ravel(),
                           (g_a + lambda_a).ravel(), (-g_a + lambda_a).ravel()])
    return obj, grad
```

A.2 Verifying a number at the precision it was written (`backend/agents/nl_reporter.py`).

```
def _close(value: float, allowed: float, decimals: int = 2) -> bool:
    # 3.97 may be written 3.97, 4.0 or 4, but never 3.78 or 7.03
    step = 0.5 * 10 ** (-decimals)
    return abs(value - allowed) <= step + abs(allowed) * 1e-3 + 1e-9
```

A.3 Idempotent, incremental storage (`backend/db.py`).

```
INSERT INTO metrics (source, metric_name, timestamp, value)
SELECT source, metric_name, timestamp, value FROM _kairos_batch
ON CONFLICT (source, metric_name, timestamp)
DO UPDATE SET value = excluded.value, ingested_at = now()
```

A.4 Writing a Fast Transfer chunk (`backend/services/transfer.py`). Parallel writers touch disjoint byte ranges; a chunk is logged as received only after it is on disk.

```
raw = _decompress(data, meta['compression'])
if chunk_sha256 and hashlib.sha256(raw).hexdigest() != chunk_sha256.lower():
    raise TransferError(f'chunk {index}: checksum mismatch, please resend')
with open(self._data(tid), 'r+b') as fh:
    fh.seek(index * meta['chunk_size'])
    fh.write(raw); fh.flush(); os.fsync(fh.fileno())
with lock, open(self._dir(tid) / 'received.log', 'a') as log:
    log.write(f'{index}\n')
```

Appendix B: User Manual

B.1 Installation

```
git clone <repository> kairos && cd kairos
python -m venv backend/venv && backend\venv\Scripts\activate
pip install -r backend/requirements.txt
cd frontend && npm install && npm run build && cd ..
copy .env.example .env      # add GROQ_API_KEY, TELEGRAM_BOT_TOKEN (optional)
uvicorn backend.main:app     # dashboard and API on http://127.0.0.1:8000
```

B.2 Using the dashboard

1. Open the **Data Sources** page and click **Add source**. Choose a type (SQL, file, SFTP/FTP, REST or push), fill in the connection details and click **Test & preview**. Apply the mapping suggested by SLAC and save.
2. Click **Sync all** to collect data. The table shows rows and bytes moved for each source.
3. On the **Overview** page click **Run pipeline now**, or wait for the 15-minute schedule. Click any anomaly to see its chart.
4. Open **Causal Analysis** to see the CBMG and the ranked root-cause chains. Hover a chain to highlight it.
5. Open **Reports & Alerts** to read the verified report and the alert history.
6. For large files, use **Fast Transfer**: drop the file, choose streams and compression, start, and click **Ingest** when it completes.

B.3 Edge agent (run next to large data inside the company network)

```
python edge_agent/kairos_edge.py aggregate --input orders.csv.gz --ts-col created_at \
    --metric amount:revenue:sum --count-metric orders --freq 1h \
    --source store_42 --server https://kairos.example.com
python edge_agent/kairos_edge.py transfer --file dump.parquet --workers 8 --compress zstd --ingest
```

Appendix C: Configuration and API Reference

C.1 Example source configuration (`sources.yaml`). Secrets are read from environment variables, never stored in the file.

```
sources:
  - name: erp_orders
    type: sql
    url: postgresql://readonly:${ERP_PASS}@erp-db/erp
    query: |
      SELECT date_trunc('hour', created_at) AS timestamp,
             SUM(amount) AS daily_revenue, COUNT(*) AS order_count
      FROM orders WHERE created_at >= :since GROUP BY 1
  - name: logistics_ftp
    type: sftp
    host: ftp.partner.com
    username: kairos_readonly
    password_env: FTP_PASS
    folder: /exports/kpis
    pattern: 'logistics_*.csv.gz'
```

C.2 Main API endpoints (base path `/api/v1`; interactive documentation at `/docs`).

Method and path	Purpose
POST /pipeline/run	Run one full monitoring cycle
GET /anomalies	List stored anomalies, filterable by severity, source and metric
POST /investigate	Run causal discovery on recent anomalies
POST /report	Generate and verify an executive report
GET, POST /sources	List or register data sources
POST /sources/{name}/sync	Incremental sync of one source
POST /ingest	Push records from a webhook or the edge agent
POST /transfer/start, PUT /transfer/{id}/chunk/{n}	Fast Transfer upload

Appendix D: Additional Results

D.1 Reproducing the results. All figures and tables in Chapter 4 are produced by the following commands. Random seeds are fixed, so the numbers repeat exactly on the same machine; timings vary with hardware.

```
python benchmark/evaluate_models.py --seeds 10       # Tables 11-13
python benchmark/benchmark_ingestion.py --rows 2000000 --stream-mbps 50   # Table 10
pytest tests -q                                       # 36 passed
python docs/report/make_figures.py                    # all figures
```

D.2 Ground-truth causal structure of the benchmark. Marketing chain: advertising spend → click-through rate → conversion rate → order count → daily revenue, with cart abandonment (negative) into conversion, average order value into revenue, and order count into refunds. Operations chain: inventory turnover (negative) → delivery delay → open support tickets → response time → NPS (negative) → churn (negative). Gross margin is independent. Two incidents were injected in the last 24 hours: a three-hour collapse in advertising spend and a three-hour spike in delivery delay, plus six isolated anomalies of graded severity.

D.3 Run times. On a laptop (Python 3.13, Windows 11), detection for 15 series took 0.23 s, DYNOTEARS 0.25 s, the full investigation with two bootstrap refits and DoWhy 1.6 s, and a complete five-agent cycle including an LLM call 12.4 s.